

23CSE314 Compiler Design

Lab Sheet 4

Sample Program:

```
C:\Users\exam\23219\labsheet4>java -cp .;antlr-4.13.2-complete.jar TestDecl  
(prog (decl (type int) (varlist a) ;) <EOF>)
```

Exercises:

1. Give the following inputs and check the output:

```
grammar Decl;  
prog : decl EOF ;  
decl : type varlist ';' ;  
type : 'int' ;  
varlist : ID (',' ID)* ;  
ID : [a-zA-Z][a-zA-Z0-9]* ;  
WS : [ \t\r\n]+ -> skip ;
```

a. int a,b,c;

```
C:\Users\exam\23219\labsheet4>java -cp .;antlr-4.13.2-complete.jar TestDecl  
(prog (decl (type int) (varlist a , b , c) ;) <EOF>)
```

b. float a, b;

```
C:\Users\exam\23219\labsheet4>java -cp .;antlr-4.13.2-complete.jar TestDecl  
line 1:0 missing 'int' at 'float'  
line 1:6 mismatched input 'a' expecting {';', ',', '}'  
(prog (decl (type <missing 'int'>) (varlist float) a , b ;) <EOF>)
```

2. Extend the same grammar to accept the declarations of the form:

```
grammar Decl;  
prog : decl EOF ;  
decl : type varlist ';' ;  
type : 'int' | 'float' | 'char' ;  
varlist : ID (',' ID)* ;  
ID : [a-zA-Z][a-zA-Z0-9]* ;  
WS : [ \t\r\n]+ -> skip ;
```

i. float a,b,sum;

```
C:\Users\exam\23219\labsheet4>java -cp .;antlr-4.13.2-complete.jar TestDecl
(prog (decl (type float) (varlist a , b , sum) ;) <EOF>)
```

ii. char ch,ch1;

```
C:\Users\exam\23219\labsheet4>java -cp .;antlr-4.13.2-complete.jar TestDecl
(prog (decl (type char) (varlist ch , ch1) ;) <EOF>)
```

3. Extend the grammar to add initialization along with declaration

```
grammar Decl;

prog      : decl EOF ;
decl      : type varlist ';' ;
type      : 'int' | 'float' | 'char' ;
varlist   : init (',' init)* ;
init      : ID ('=' value)? ;
value     : INT | FLOAT | CHAR ;
ID        : [a-zA-Z][a-zA-Z0-9]* ;
INT       : [0-9]+ ;
FLOAT     : [0-9]+ '.' [0-9]+ ;
CHAR      : '\'' [a-zA-Z0-9] '\'' ;
WS        : [ \t\r\n]+ -> skip ;
```

int a = 10;

```
C:\Users\exam\23219\labsheet4>java -cp .;antlr-4.13.2-complete.jar TestDecl
(prog (decl (type int) (varlist (init a = (value 10)))) ;) <EOF>)
```

float x = 2.5;

```
C:\Users\exam\23219\labsheet4>java -cp .;antlr-4.13.2-complete.jar TestDecl
(prog (decl (type float) (varlist (init x = (value 2.5)))) ;) <EOF>)
```

4. Extend the grammar to allow Multiple Declarations in One Line

int a, b, c; float x, y

```
grammar Decl;

prog      : decl+ EOF ;
decl      : type varlist ';' ;
type      : 'int' | 'float' | 'char' ;
varlist   : init (',' init)* ;
init      : ID ('=' value)? ;
value     : INT | FLOAT | CHAR ;
ID        : [a-zA-Z][a-zA-Z0-9]* ;
INT       : [0-9]+ ;
FLOAT     : [0-9]+ '.' [0-9]+ ;
CHAR      : '\'' [a-zA-Z0-9] '\'' ;
WS        : [ \t\r\n]+ -> skip ;
```

```
C:\Users\exam\23219\labsheet4>java -cp .;antlr-4.13.2-complete.jar TestDecl
(prog (decl (type int) (varlist (init a) , (init b) , (init c)) ;) (decl (type float)
(varlist (init x) , (init y)) ;) <EOF>)
```

5. Extend the syntax rules to accept declaration statements of the form:

```
grammar Decl;
prog      : decl+ EOF ;
decl      : 'var' type varlist ';' ;
type      : 'int' | 'float' | 'char' | 'string' ;
varlist   : init (',' init)* ;
init      : ID ('=' value)? ;
value     : INT | FLOAT | CHAR | STRING ;
ID        : [a-zA-Z][a-zA-Z0-9]* ;
INT       : [0-9]+ ;
FLOAT     : [0-9]+ '.' [0-9]+ ;
CHAR      : '\'' [a-zA-Z0-9] '\'' ;
STRING    : '"' (~["\r\n])* '"' ;
WS        : [ \t\r\n]+ -> skip ;
```

i. var int tokens = 2, tip;

```
C:\Users\exam\23219\labsheet4>java -cp .;antlr-4.13.2-complete.jar TestDecl
(prog (decl var (type int) (varlist (init tokens = (value 2)) , (init tip)) ;) <EOF>)
```

ii. var string xyz = "hello";

```
C:\Users\exam\23219\labsheet4>java -cp .;antlr-4.13.2-complete.jar TestDecl
(prog (decl var (type string) (varlist (init xyz = (value "hello")))) ;) <EOF>)
```

iii. Extend the lexical and syntax rules to recognize floating point numbers (2.345) as FLOAT and in declaration statement.

Example: var float time = 12.34

```
C:\Users\exam\23219\labsheet4>java -cp .;antlr-4.13.2-complete.jar TestDecl
(prog (decl var (type float) (varlist (init time = (value 12.34)))) ;) <EOF>)
```

6. Constant Declaration: Extend the syntax rules to recognize declaration of constants of the form: const int pi = 3.14;

```
grammar Decl;
prog      : decl+ EOF ;
decl      : ('var' | 'const') type varlist ';' ;
type      : 'int' | 'float' | 'char' | 'string' ;
varlist   : init (',' init)* ;
init      : ID ('=' value)? ;
value     : INT | FLOAT | CHAR | STRING ;
ID        : [a-zA-Z][a-zA-Z0-9]* ;
INT       : [0-9]+ ;
FLOAT     : [0-9]+ '.' [0-9]+ ;
CHAR      : '\'' [a-zA-Z0-9] '\'' ;
STRING    : '"' (~["\r\n])* '"' ;
WS        : [ \t\r\n]+ -> skip ;
```

```
C:\Users\exam\23219\labsheet4>java -cp .;antlr-4.13.2-complete.jar TestDecl
(prog (decl const (type int) (varlist (init pi = (value 3.14)))) ;) <EOF>)
```

7. NULL value: Extend your syntax rules to accept nil as a value during declaration

Example: var int i = NULL; var string s = NULL;

```
grammar Decl;
prog      : decl+ EOF ;
decl      : ('var' | 'const') type varlist ';' ;
type      : 'int' | 'float' | 'char' | 'string' ;
varlist   : init (',' init)* ;
init      : ID ('=' value)? ;
value     : INT | FLOAT | CHAR | STRING | NULL;
NULL      : 'NULL';
ID        : [a-zA-Z][a-zA-Z0-9]* ;
INT       : [0-9]+ ;
FLOAT     : [0-9]+ '.' [0-9]+ ;
CHAR      : '\'' [a-zA-Z0-9] '\'' ;
STRING    : '"' (~["\r\n])* '"' ;
WS        : [ \t\r\n]+ -> skip ;
```

```
C:\Users\exam\23219\labsheet4>java -cp .;antlr-4.13.2-complete.jar TestDecl
(prog (decl var (type int) (varlist (init i = (value NULL)))) ;) (decl var (type string
) (varlist (init s = (value NULL)))) ;) <EOF>)
```

8. Array Declaration: Extend your rules to recognize array declarations of the form: int

value[10]; string colours[5];

```
grammar Decl;
prog      : decl+ EOF ;
decl      : ('var' | 'const')? type varlist ';' ;
type      : 'int' | 'float' | 'char' | 'string' ;
varlist   : init (',' init)* ;
init      : ID array? ('=' value)? ;
array     : '[' INT ']';
value     : INT | FLOAT | CHAR | STRING | NULL ;
NULL      : 'NULL';
ID        : [a-zA-Z][a-zA-Z0-9]* ;
INT       : [0-9]+ ;
FLOAT     : [0-9]+ '.' [0-9]+ ;
CHAR      : '\'' [a-zA-Z0-9] '\'' ;
STRING    : '"' (~["\r\n])* '"' ;
WS        : [ \t\r\n]+ -> skip ;
```

```
C:\Users\exam\23219\labsheet4>java -cp .;antlr-4.13.2-complete.jar TestDecl
(prog (decl (type int) (varlist (init value (array [ 10 ]))) ;) (decl (type string)
(varlist (init colours (array [ 5 ]))) ;) <EOF>)
```

9. Array Initialization: Extend your rules to recognize array initializations of the form:

```
grammar Decl;
prog      : decl+ EOF ;
decl      : ('var' | 'const')? type varlist ';' ;
type      : 'int' | 'float' | 'char' | 'string' ;
varlist   : init (',' init)* ;
init      : ID array? ('=' (value | arrayInit) )? ;
array     : '[' INT ']' ;
arrayInit : '{' value (',' value)* '}' ;

value     : INT | FLOAT | CHAR | STRING | NULL ;
NULL      : 'NULL' ;
ID        : [a-zA-Z][a-zA-Z0-9]* ;
INT       : [0-9]+ ;
FLOAT     : [0-9]+ '.' [0-9]+ ;
CHAR      : '\'' [a-zA-Z0-9] '\'' ;
STRING    : '"' (~["\r\n])* '"' ;
WS       : [ \t\r\n]+ -> skip ;
```

```
int value[10] = {1, 25, 43, .....};
```

```
C:\Users\exam\23219\labsheet4>java -cp .;antlr-4.13.2-complete.jar TestDecl
(prog (decl (type int) (varlist (init value (array [ 10 ])) = (arrayInit { (value 1)
, (value 25) , (value 43) }))) ;) <EOF>)
```

```
string colors[5] = {"red", "green", "blue", "orange", "yellow"}
```

```
C:\Users\exam\23219\labsheet4>java -cp .;antlr-4.13.2-complete.jar TestDecl
(prog (decl (type string) (varlist (init colors (array [ 5 ])) = (arrayInit { (value "red")
, (value "green") , (value "blue") , (value "orange") , (value "yellow") }))) ;) <EOF>)
```